

Informe técnico final · Unidades 4 · 5 · 6

ParkControl: validación y gestión de defectos

Síntesis del proceso de pruebas: integración continua, rendimiento bajo carga y ciclo de vida del defecto.

INTEGRANTES · GRUPO 9

Yerlinson Maturana Serna

Sadane Geronimo Miguel Santiago Acevedo Virgues

Brayan Estif Calderón Gómez

Julián Camilo Corredor Rojas

SISTEMA

ParkControl · Node.js

Una estrategia de validación multinivel, automatizada de extremo a extremo

ParkControl registra ingresos y salidas, calcula tarifas y emite facturas. Lo sometimos a cinco niveles de prueba complementarios.

01

Unitarias

Dominio, tarifas y persistencia

02

Integración

Módulos trabajando juntos

03

Servicio HTTP

Endpoints reales de la API

04

E2E

Flujo de interfaz (Playwright)

05

Carga

Rendimiento con JMeter

META U6 Gestionar el ciclo de vida del defecto, priorizar con métricas de calidad y comunicar los hallazgos de forma técnica y profesional.

La cobertura al 100 % es el gate que bloquea el merge

Cada push y cada pull request ejecuta toda la suite en GitHub Actions; la cobertura actúa como puerta de calidad antes de integrar a la rama principal.



npm test · coverage

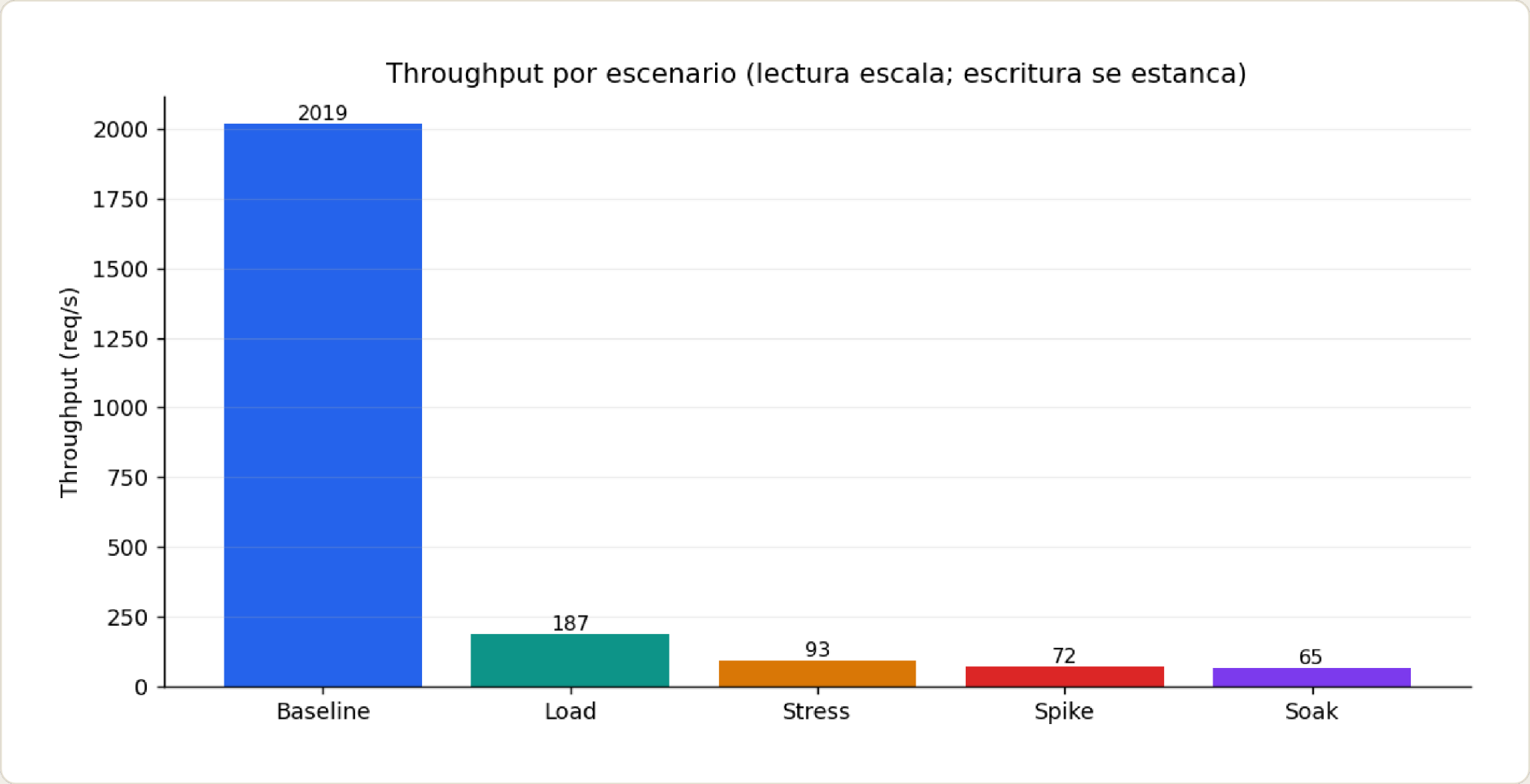
ParkControl - pruebas automatizadas y cobertura (CI)

```
$ npm test
# tests 33
# suites 6
# pass 33
# fail 0
# cancelled 0
# skipped 0
# todo 0

$ npm run coverage
# start of coverage report
# -----
# file | line % | branch % | funcs % | uncovered lines
# -----
# src
# domain
#   parkingLot.js | 100.00 | 100.00 | 100.00 |
#   tariffCalculator.js | 100.00 | 100.00 | 100.00 |
# server
#   app.js | 100.00 | 100.00 | 100.00 |
#   httpUtils.js | 100.00 | 100.00 | 100.00 |
#   storage
#   jsonStore.js | 100.00 | 100.00 | 100.00 |
# -----
# all files | 100.00 | 100.00 | 100.00 |
# -----
# end of coverage report
```

La lectura escala; la escritura se desploma con la carga

Cinco escenarios con JMeter, sin un solo fallo. El contraste aparece en cómo responde cada operación al aumentar los usuarios concurrentes.



25.462
muestras JMeter

0 %
tasa de error

THROUGHPUT DE ESCRITURA

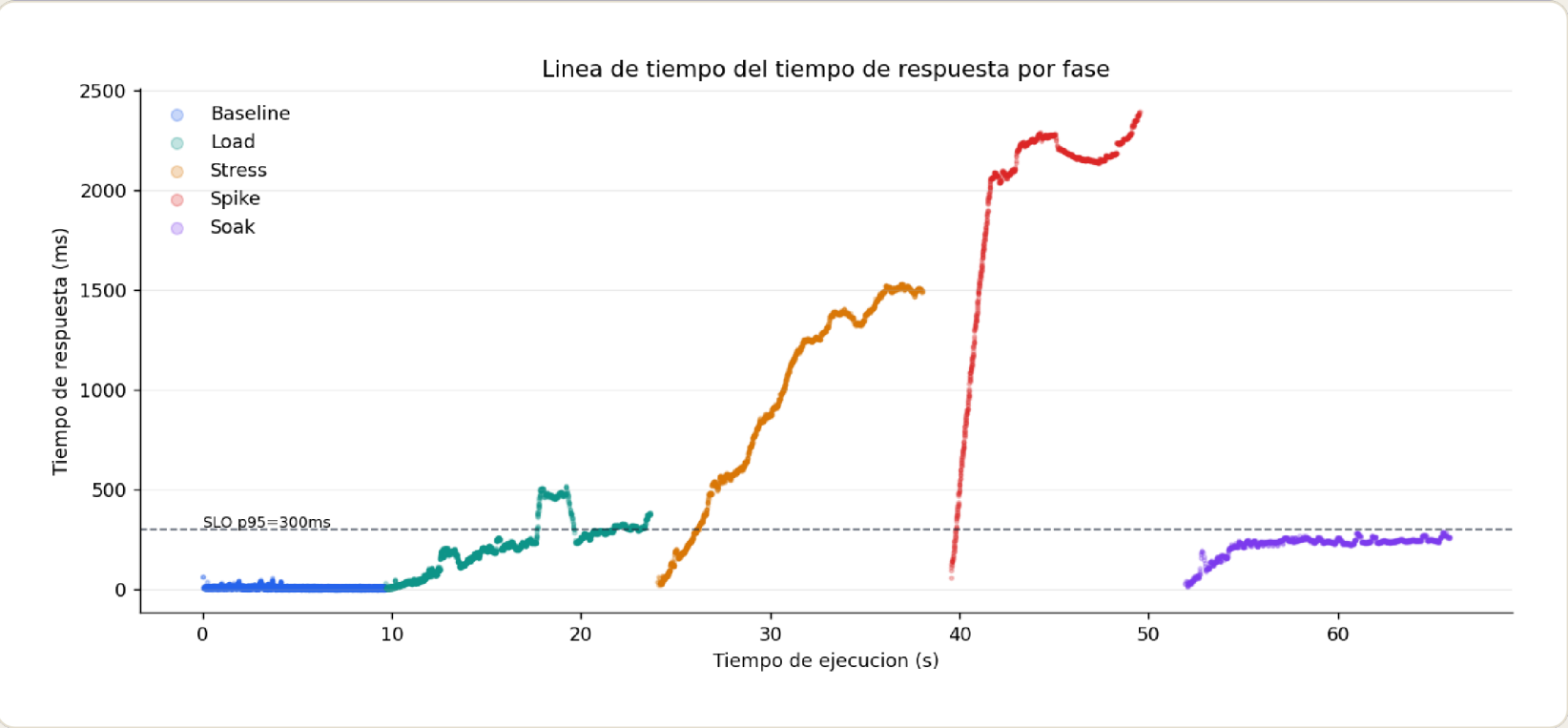
187 → 93 → 72 → 65 req/s

No se mantiene constante: **decrece** al subir la concurrencia. La lectura, en cambio, supera 2.000 req/s.

La escritura solo cumple el SLO con baja concurrencia

Fijamos umbrales de latencia y error como criterio objetivo de aceptación: la lectura los cumple siempre; la escritura, solo a baja carga.

ESCENARIO	P95	SLO P95	USUARIOS
Baseline	11 ms	CUMPLE	10 · lectura
Load	447 ms	INCUMPLE	40 · escritura
Stress	1.506 ms	INCUMPLE	120 · escritura
Spike	2.280 ms	INCUMPLE	160 · escritura
Soak	264 ms	CUMPLE	16 · escritura



SLO: **p95 < 300 ms · p99 < 800 ms · error < 1 %**. En Stress la latencia crece de forma sostenida — la firma de un archivo que no para de crecer.

La causa raíz: dos cuellos de botella compuestos

El comportamiento no es aleatorio: dos decisiones de diseño se combinan y degradan la escritura a medida que crecen los datos y la concurrencia.

CUELLO 1

Persistencia $O(n)$

Cada escritura reescribe todo el archivo JSON.

El historial nunca se purga: el archivo crece y el costo por operación aumenta con el tiempo.

CUELLO 2

Escritura serializada

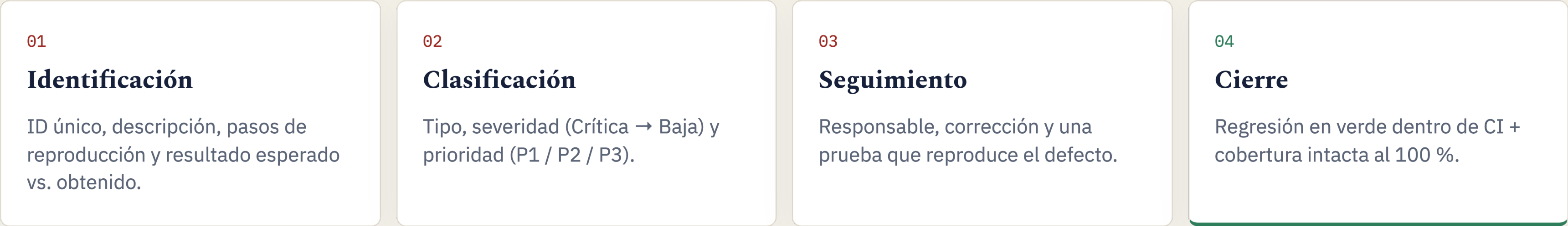
Una única cola procesa una operación a la vez.

Bajo concurrencia los usuarios se encolan: la latencia crece en proporción a la carga.

EVIDENCIA Soak con 16 usuarios rinde *peor* que Load con 40 — porque corre sobre un archivo ya mucho más grande.

Cuatro etapas — un defecto se cierra solo con su prueba en verde

Cada defecto recorre un flujo trazable y documentado; el cierre exige evidencia automatizada, no la sola opinión del desarrollador.



11 defectos gestionados, priorizados por impacto técnico

Clasificamos por severidad y prioridad para decidir qué corregir primero según su impacto técnico real.

11

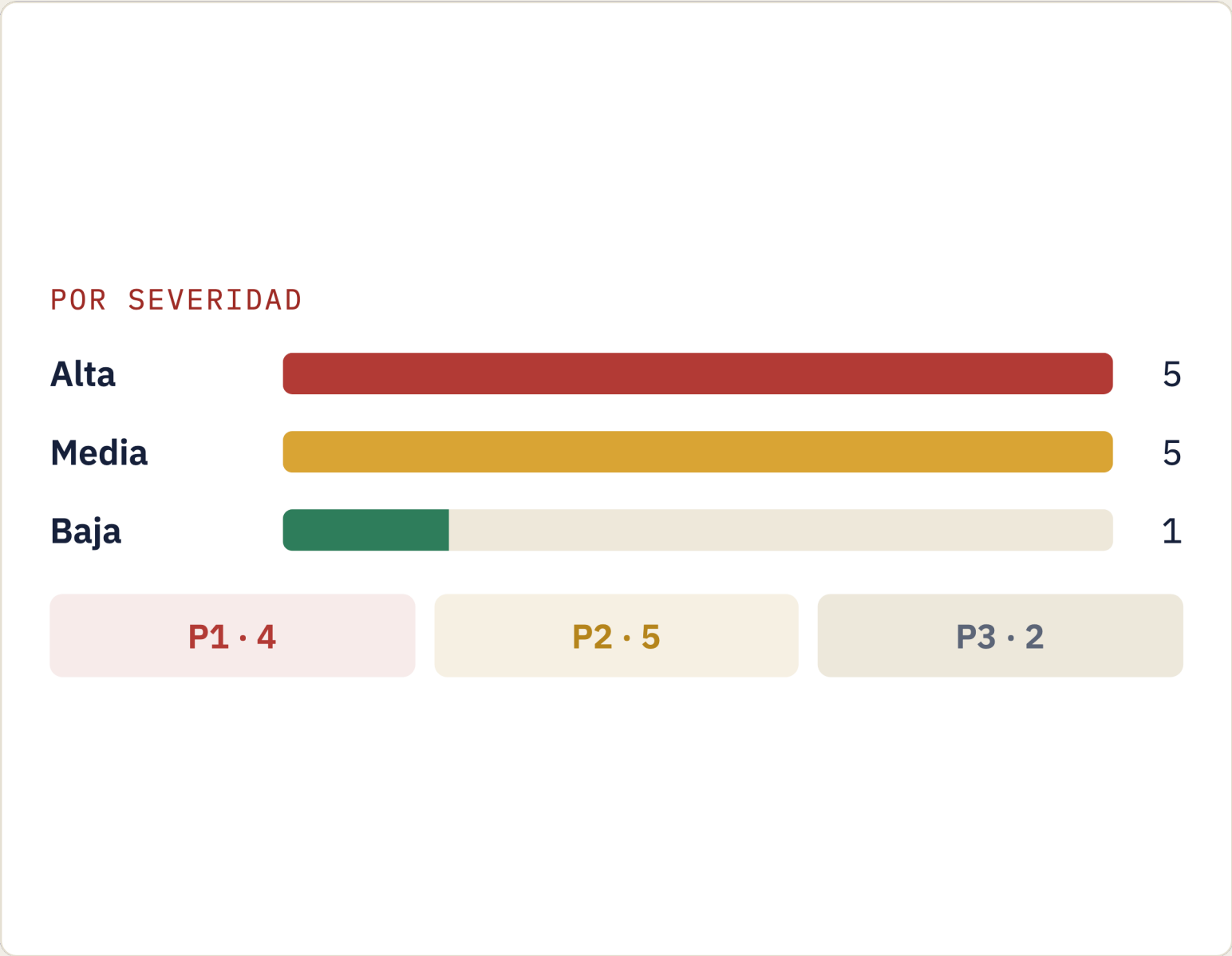
totales

6

cerrados

5

abiertos



BACKLOG ABIERTO · POR PRIORIDAD

P1

DEF-07

Degradación de throughput (O(n))
Rendimiento

P2

DEF-08

Escritura serializada limita concurrencia
Rendimiento

P2

DEF-10

Consecutivo de factura no reinicia por año
Funcional / contable

P3

DEF-11

Historial del dashboard sin paginar
Rendimiento

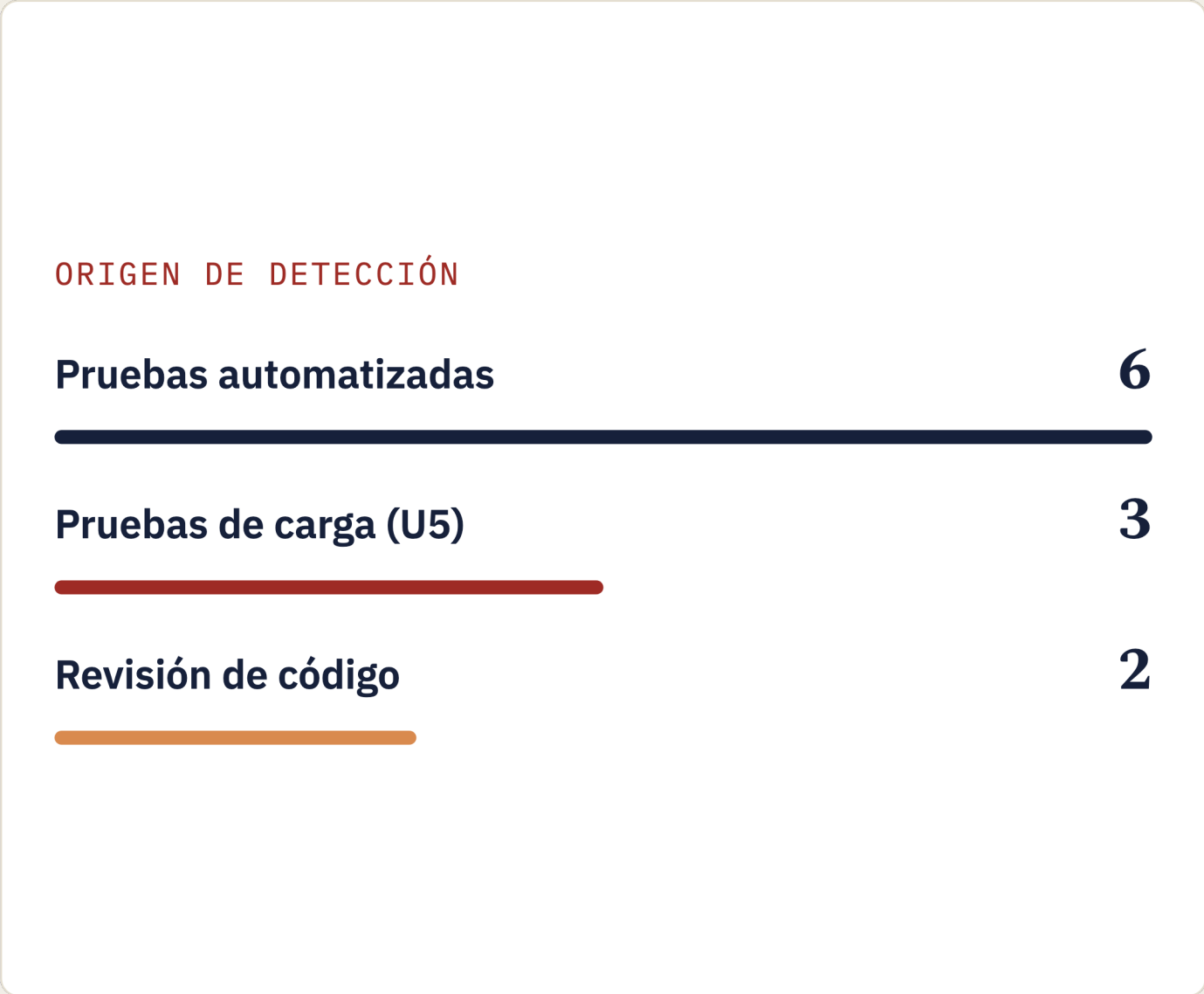
P3

DEF-09

Placa de solo guiones aceptada
Validación

Cada defecto es trazable: requisito, prueba y estado

Los defectos provienen de distintas técnicas de prueba; cada uno enlaza su requisito, su caso de prueba y su estado actual.



Backlog de mejora, ordenado por impacto

Convertimos los hallazgos en un plan accionable, ordenado de mayor a menor impacto sobre la calidad del sistema.

1	Migrar a base de datos transaccional Escritura incremental y concurrencia controlada — elimina la causa raíz O(n).	Impacto alto · DEF-07/08
2	Paginar y archivar el historial Sostiene el alto throughput de lectura aun con datos grandes.	P3 · DEF-11
3	Numerar la factura por año fiscal Corrección contable simple — el consecutivo debe reiniciar cada año.	P2 · DEF-10
4	Endurecer la validación de placa Exigir al menos un carácter alfanumérico (rechazar «-----»).	P3 · DEF-09

Cada corrección entra al pipeline con su **prueba de regresión**; se suma una prueba de rendimiento que falle si el throughput de escritura cae de un umbral.

Cada nivel de prueba reveló algo distinto

FUNCIONAL

Muy efectiva

El gate de cobertura cerró 6 defectos antes de integrarse a main.

CARGA

Reveladora

3 defectos solo emergieron bajo carga: lo funcional no basta.

REVISIÓN

Complementaria

2 casos de borde que las pruebas no cubrían — ahora regresión.

Sistema correcto, límites de escalabilidad cuantificados y un backlog técnico claro, priorizado y trazable para la siguiente iteración.